DEPARTMENT OF MATHEMATICS, IIT MADRAS

MA5950 — Mathematical Finance: Theory & Implementation

Instructor: Prof. Neelesh Upadhye

Lecture 16: Multi-Period Binomial Trees

Recombining Lattices · Backward Induction · Convergence

September 7, 2026

Today's Agenda

1. From one coin flip to many

Today's Agenda

1. From one coin flip to many
2. Why the tree recombines

Today's Agenda

1. From one coin flip to many
2. Why the tree recombines
3. Backward induction

Today's Agenda

1. From one coin flip to many
2. Why the tree recombines
3. Backward induction
4. The CRR parameterization

Today's Agenda

1. From one coin flip to many
2. Why the tree recombines
3. Backward induction
4. The CRR parameterization
5. Convergence as N grows

Learning Objectives

- Build an N-step recombining lattice.

Learning Objectives

- Build an N-step recombining lattice.
- Price a European option by backward induction.

Learning Objectives

- Build an N-step recombining lattice.
- Price a European option by backward induction.
- Explain why recombination matters computationally.

Learning Objectives

- Build an N-step recombining lattice.
- Price a European option by backward induction.
- Explain why recombination matters computationally.
- Describe convergence as the number of steps grows.

A Staircase, Not a Single Leap

Breaking Time Into Steps

Pricing a two-year option in one giant step is like calling a staircase a single jump.

A Staircase, Not a Single Leap

Breaking Time Into Steps

Pricing a two-year option in one giant step is like calling a staircase a single jump. Break it into many small steps and the paths fan out realistically.

A Staircase, Not a Single Leap

Breaking Time Into Steps

Pricing a two-year option in one giant step is like calling a staircase a single jump. Break it into many small steps and the paths fan out realistically. Crucially: no new theory needed – each step is exactly Week 5's one-period problem.

The N-Step Lattice

Definition

After n steps with $n - j$ ups and j downs:

$$S_{n,j} = S_0 \, u^{n-j} \, d^j, \qquad j = 0, \ldots, n$$

The N-Step Lattice

Definition

After n steps with $n - j$ ups and j downs:

$$S_{n,j} = S_0\, u^{n-j}\, d^{j}, \qquad j = 0, \ldots, n$$

Why it recombines

"up-then-down" $= S_0 ud$; "down-then-up" $= S_0 du$.

The N-Step Lattice

Definition

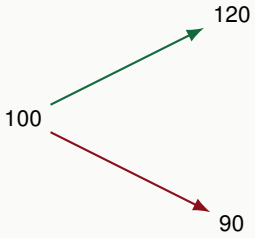
After n steps with $n - j$ ups and j downs:

$$S_{n,j} = S_0\, u^{n-j}\, d^{j}, \qquad j = 0, \ldots, n$$

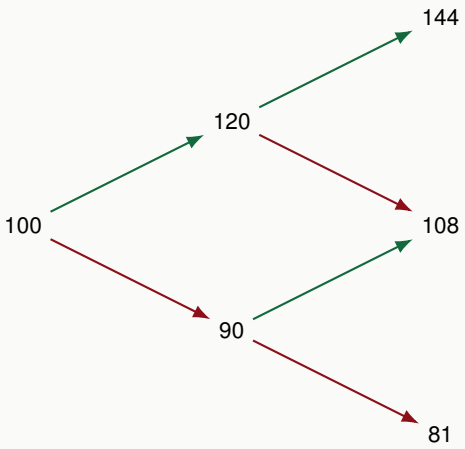
Why it recombines

"up-then-down" $= S_0 ud$; "down-then-up" $= S_0 du$. **Same value.** Only the *count* of ups and downs matters, not the order.

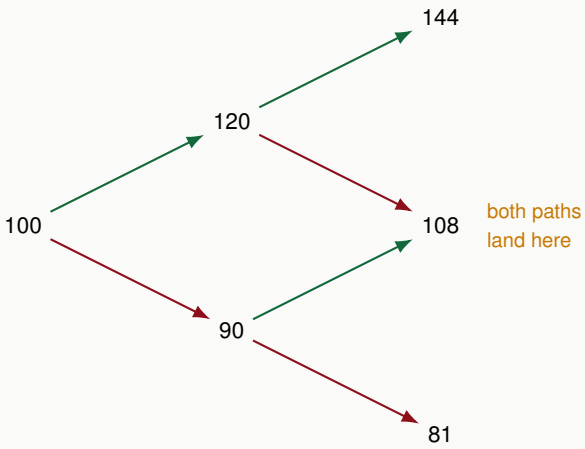
A Two-Step Recombining Lattice



A Two-Step Recombining Lattice



A Two-Step Recombining Lattice



Why Recombination Matters

Key Takeaway

Without recombination: 2^N end nodes.

Why Recombination Matters

Key Takeaway

Without recombination: 2^N end nodes. $2^{50} \approx 10^{15}$ – hopeless.

Why Recombination Matters

Key Takeaway

Without recombination: 2^N end nodes. $2^{50} \approx 10^{15}$ – hopeless. With recombination: step n has just $n + 1$ nodes.

Why Recombination Matters

Key Takeaway

Without recombination: 2^N end nodes. $2^{50} \approx 10^{15}$ – hopeless. With recombination: step n has just $n+1$ nodes. For N=50: only 1326 nodes total.

This single structural fact makes binomial pricing practical.

The Recursion

Backward Induction

$$V_{N,j} = \text{payoff}(S_{N,j})$$

$$V_{n,j} = \frac{1}{1+r}\Big[q\, V_{n+1,j} + (1-q)\, V_{n+1,j+1}\Big]$$

The Recursion

Backward Induction

$$V_{N,j} = \text{payoff}(S_{N,j})$$

$$V_{n,j} = \frac{1}{1+r}\Big[q\, V_{n+1,j} + (1-q)\, V_{n+1,j+1}\Big]$$

Work from $n = N - 1$ back to 0. Then $V_{0,0}$ is today's price.

Solving a Maze From the Exit

Why Compute Backwards?

Strange at first – we compute the future before the present.

Solving a Maze From the Exit

Why Compute Backwards?

Strange at first – we compute the future before the present. But it's how you solve a maze efficiently: start where the answer is obvious (the exit), work back.

Solving a Maze From the Exit

Why Compute Backwards?

Strange at first – we compute the future before the present. But it's how you solve a maze efficiently: start where the answer is obvious (the exit), work back. Maturity is our "exit": the payoff there is certain, requiring no modeling at all.

Worked Example: Setting Up

S_0=100, u=1.2, d=0.9, r=5%, K=105, N=2

$$q = \frac{1.05 - 0.9}{1.2 - 0.9} = 0.5$$

Worked Example: Setting Up

$S_0=100$, $u=1.2$, $d=0.9$, $r=5\%$, $K=105$, $N=2$

$$q = \frac{1.05 - 0.9}{1.2 - 0.9} = 0.5$$

Terminal payoffs at $t=2$:

$$V_{2,0} = \max(144-105, 0) = 39, \quad V_{2,1} = 3, \quad V_{2,2} = 0$$

Worked Example: Rolling Back

Roll back to t=1:

$$V_{1,0} = \frac{1}{1.05}[0.5(39) + 0.5(3)] = 20$$

Worked Example: Rolling Back

Roll back to t=1:

$$V_{1,0} = \frac{1}{1.05}[0.5(39) + 0.5(3)] = 20$$

$$V_{1,1} = \frac{1}{1.05}[0.5(3) + 0.5(0)] \approx 1.4286$$

Worked Example: Rolling Back

Roll back to t=1:

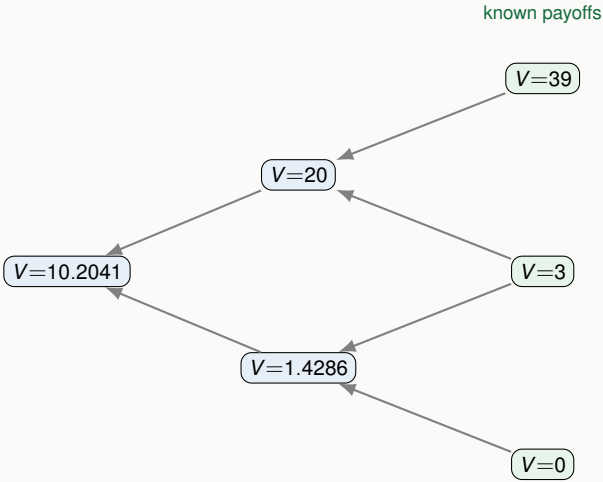
$$V_{1,0} = \frac{1}{1.05}[0.5(39) + 0.5(3)] = 20$$

$$V_{1,1} = \frac{1}{1.05}[0.5(3) + 0.5(0)] \approx 1.4286$$

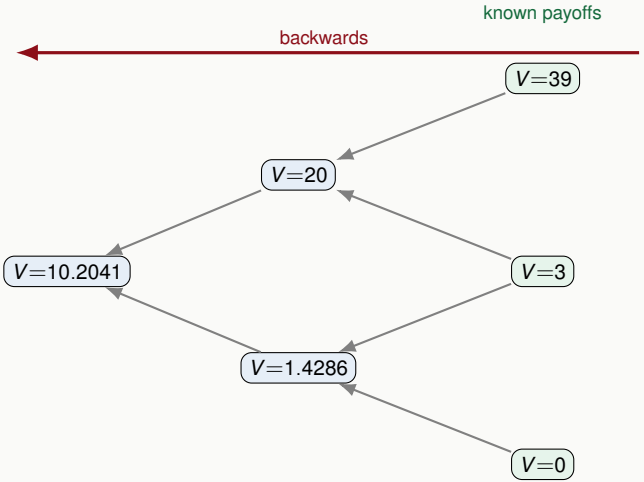
Roll back to t=0:

$$V_{0,0} = \frac{1}{1.05}[0.5(20) + 0.5(1.4286)] \approx \mathbf{10.2041}$$

The Direction of Computation



The Direction of Computation



Every arrow-pair is one application of Week 5's one-period formula.

Choosing u and d

Cox–Ross–Rubinstein

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad q = \frac{e^{r\Delta t} - d}{u - d}$$

with $\Delta t = T/N$.

Choosing u and d

Cox–Ross–Rubinstein

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad q = \frac{e^{r\Delta t} - d}{u - d}$$

with $\Delta t = T/N$.

Why $d = 1/u$?

Guarantees symmetric recombination about S_0.

Choosing u and d

Cox–Ross–Rubinstein

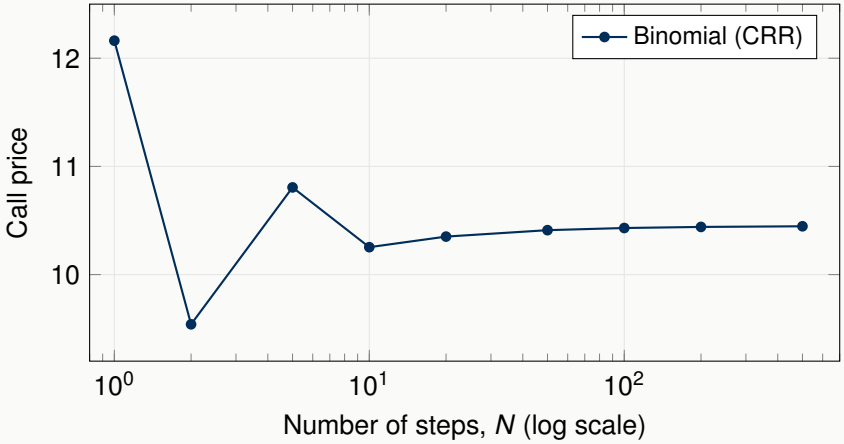
$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad q = \frac{e^{r\Delta t} - d}{u - d}$$

with $\Delta t = T/N$.

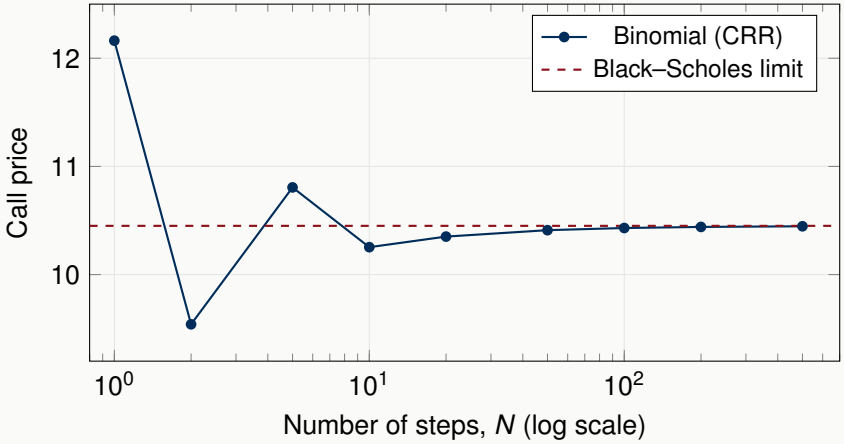
Why $d = 1/u$?

Guarantees symmetric recombination about S_0. Note the switch to continuous compounding $e^{r\Delta t}$ – natural once steps are short (Week 1, Lecture 2).

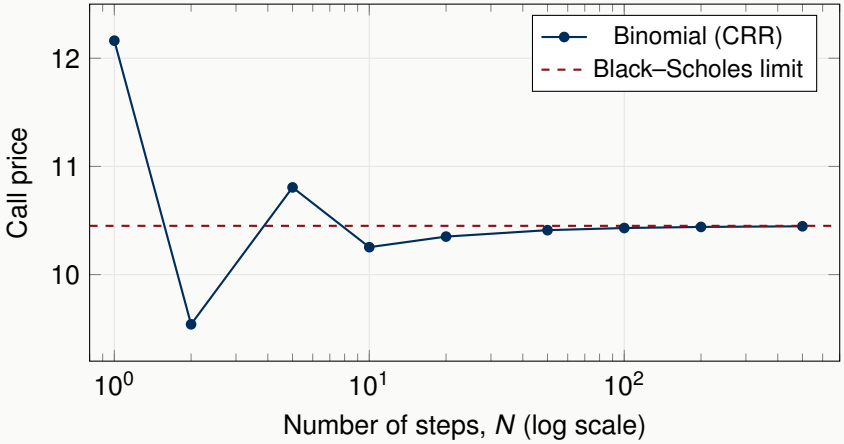
Convergence as N Grows



Convergence as N Grows



Convergence as N Grows



$S_0 = K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$yr.

What the Picture Shows

Key Takeaway

- Converges to the **Black–Scholes price** (≈ 10.4506) – derived properly in Week 9.

What the Picture Shows

Key Takeaway

- Converges to the **Black–Scholes price** (≈ 10.4506) – derived properly in Week 9.
- Convergence is **oscillatory**, not monotone.

What the Picture Shows

Key Takeaway

- Converges to the **Black–Scholes price** (≈ 10.4506) – derived properly in Week 9.
- Convergence is **oscillatory**, not monotone.
- Error roughly *halves* when N doubles: order $1/N$.

Pixels Approximating a Curve

The Binomial Tree as Low Resolution

Few steps = a blocky, low-res image of a smooth curve.

Pixels Approximating a Curve

The Binomial Tree as Low Resolution

Few steps = a blocky, low-res image of a smooth curve. More steps = sharper picture.

Pixels Approximating a Curve

The Binomial Tree as Low Resolution

Few steps = a blocky, low-res image of a smooth curve. More steps = sharper picture. Black–Scholes is the infinitely-fine picture; the binomial model is the pixelated approximation.

Pixels Approximating a Curve

The Binomial Tree as Low Resolution

Few steps = a blocky, low-res image of a smooth curve. More steps = sharper picture. Black–Scholes is the infinitely-fine picture; the binomial model is the pixelated approximation. Week 9 will feel less like new theory, more like the natural destination.

Key Takeaways

Key Takeaway

- $S_{n,j} = S_0 u^{n-j} d^j$; the tree **recombines** ($n+1$ nodes, not 2^n).

Key Takeaways

Key Takeaway

- $S_{n,j} = S_0 u^{n-j} d^j$; the tree **recombines** ($n+1$ nodes, not 2^n).
- **Backward induction**: terminal payoffs, then roll back with the one-period formula.

Key Takeaways

Key Takeaway

- $S_{n,j} = S_0 u^{n-j} d^j$; the tree **recombines** ($n+1$ nodes, not 2^n).
- **Backward induction**: terminal payoffs, then roll back with the one-period formula.
- Each rollback is exactly Week 5 – no new principle.

Key Takeaways

Key Takeaway

- $S_{n,j} = S_0 u^{n-j} d^j$; the tree **recombines** ($n+1$ nodes, not 2^n).
- **Backward induction**: terminal payoffs, then roll back with the one-period formula.
- Each rollback is exactly Week 5 – no new principle.
- **CRR**: $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$ matches observed volatility.

Key Takeaways

Key Takeaway

- $S_{n,j} = S_0 u^{n-j} d^j$; the tree **recombines** ($n+1$ nodes, not 2^n).
- **Backward induction**: terminal payoffs, then roll back with the one-period formula.
- Each rollback is exactly Week 5 – no new principle.
- **CRR**: $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$ matches observed volatility.
- As $N \to \infty$: oscillating convergence to Black–Scholes, at order $1/N$.

Check Your Understanding

Discuss (2 minutes)

1. $S_0=50$, $u=1.1$, $d=0.9$: how many nodes at $t=2$? How many without recombination?

Check Your Understanding

Discuss (2 minutes)

1. S_0=50, u=1.1, d=0.9: how many nodes at t=2? How many without recombination?
2. Why is computing “backwards” not circular reasoning?

Check Your Understanding

Discuss (2 minutes)

1. $S_0=50$, $u=1.1$, $d=0.9$: how many nodes at $t=2$? How many without recombination?
2. Why is computing "backwards" not circular reasoning?
3. CRR with $\sigma=25\%$, $T=0.5$, $N=4$: find Δt, u, d.

Looking Ahead

Backward induction *works* – but why *should* it?

Looking Ahead

Backward induction *works* – but why *should* it?

Martingales – Lecture 17

Looking Ahead

Backward induction *works* – but why *should* it?

Martingales – Lecture 17

Backward induction is the martingale property, one node at a time.